



Phase 5

1. Synchronous vs Asynchronous

Synchronous kya hota hai?

JavaScript by default **synchronous** aur **single-threaded** hai.

- **Single-threaded** = ek time pe sirf ek hi kaam kar sakta hai. Ek hi "haath" hai jisse saara kaam hota hai.
- **Synchronous** = code line-by-line, upar se neeche, ek-ek karke chalta hai. Jab tak ek line complete nahi hoti, agli line start nahi hoti.

```
console.log("1. Chai banao");  
console.log("2. Chai piyo");  
console.log("3. Cup dho do");
```

```
// Output:  
// 1. Chai banao  
// 2. Chai piyo  
// 3. Cup dho do
```

Yahan sab kuch order me chala. Simple.

Problem kahan aati hai? Maan lo koi kaam time leta hai — jaise server se data laana, ya 5 second ka timer. Synchronous duniya me wo kaam poora hone tak **poora program ruk jaata hai (blocking)**.

```
console.log("Start");  
  
// Maan lo ye ek 5 second wala blocking kaam hai
```

```
let now = Date.now();
while (Date.now() - now < 5000) {
  // 5 second tak kuch nahi, bas wait... browser HANG ho gaya
  😵
}

console.log("End");
```

Is 5 second me browser pura freeze ho jaayega — button click nahi hoga, scroll nahi hoga, kuch nahi. **Ye blocking behaviour bahut bura hai.**

Asynchronous kya hota hai?

Asynchronous ka matlab — koi lamba kaam start kar do, usko side me chalne do, aur baaki code aage chalta rahe. Jab wo kaam complete hoga, tab uska result handle kar lenge.

Real life analogy: **Restaurant ka waiter** 🤖

- Synchronous waiter: ek table ka order leta hai, kitchen me khada rehta hai jab tak khana ready na ho, fir serve karta hai, fir agle table pe jaata hai. (Slow, sab wait karte hain.)
- Asynchronous waiter: order leta hai, kitchen ko de deta hai, aur turant doosre tables ke order lene chala jaata hai. Jab khana ready hota hai, tab serve kar deta hai.

Async waiter zyada efficient hai. JavaScript bhi yahi karta hai async kaam ke liye.

```
console.log("Start");

setTimeout(() => {
  console.log("3 second baad ye chala");
}, 3000);

console.log("End");

// Output:
// Start
```

```
// End
// (3 second baad) 3 second baad ye chala
```

Dekho — `setTimeout` ne 3 second wala kaam side me daal diya, aur code ruka nahi. "End" turant print ho gaya. **Ye non-blocking behaviour hai. Yahi async ki power hai.**

Async kab chahiye?

- Server/API se data laana (network requests)
- File read/write karna
- Timers (`setTimeout` , `setInterval`)
- Database operations
- User ke kisi action ka wait karna

Yaad rakho: JS khud single-threaded hai, par browser (ya Node.js) async kaam ko side me handle karne me JS ki madad karta hai. Kaise — wo aage Event Loop wale section me detail me dekhenge. 🔥

2. Callbacks

Callback kya hai?

Callback = ek function jo doosre function ko **argument ke roop me** pass kiya jaata hai, taaki baad me uss function ko "call back" (bula) kiya ja sake.

Simple words me: "Ye kaam ho jaaye, toh ye wala function chala dena."

```
function namasteBolo(naam) {
  console.log("Namaste " + naam);
}

function welcome(callback) {
  let user = "Rahul";
  callback(user); // yahan callback ko call kiya
}
```

```
welcome(namasteBolo);  
// Output: Namaste Rahul
```

Yahan `namasteBolo` ek **callback function** hai jo `welcome` ko pass hua.

Callbacks 2 tarah ke hote hain:

- **Synchronous callbacks** — turant chalte hain (jaise `arr.map`, `arr.forEach`).
- **Asynchronous callbacks** — kisi kaam ke complete hone ke baad chalte hain (jaise `setTimeout`).

```
// Synchronous callback example  
[1, 2, 3].forEach(function (num) {  
  console.log(num * 2);  
});  
// 2, 4, 6 – turant
```

`setTimeout` — ek baar, kuch time baad

`setTimeout(function, delay)` ek function ko diye gaye **delay (milliseconds)** ke baad **ek baar** chalata hai.

```
setTimeout(() => {  
  console.log("2 second ho gaye!");  
}, 2000); // 2000 ms = 2 second
```

Important point: delay ka matlab "exactly itne time baad" nahi hai. Iska matlab hai "kam se kam itne time baad". Kyon? Kyunki callback ko queue me wait karna padta hai (Event Loop section me clear hoga).

```
setTimeout(() => console.log("Hello"), 0);  
console.log("World");  
  
// Output:  
// World
```

```
// Hello
// (Haan! 0 ms ke baad bhi "World" pehle aata hai 😬 — iska reason event loop hai)
```

Arguments pass karna ho callback ko:

```
function greet(naam, sheher) {
  console.log(`Hi ${naam} from ${sheher}`);
}

setTimeout(greet, 1000, "Priya", "Indore");
// 1 second baad: Hi Priya from Indore
```

setInterval — baar-baar, repeat

`setInterval(function, delay)` ek function ko har `delay` ms par baar-baar (repeatedly) chalata rehta hai jab tak aap rokte nahi.

```
let count = 1;
setInterval(() => {
  console.log("Count: " + count);
  count++;
}, 1000);

// Har 1 second me:
// Count: 1
// Count: 2
// Count: 3
// ... infinite, jab tak roko na
```

clearTimeout aur **clearInterval** — rokne ke liye

`setTimeout` aur `setInterval` dono ek **ID return karte hain**. Iss ID ko use karke aap timer ko cancel kar sakte ho.

`clearTimeout(id)` — pending `setTimeout` ko cancel karta hai (chalne se pehle).

```
let timerId = setTimeout(() => {
  console.log("Ye kabhi print nahi hoga");
}, 3000);

clearTimeout(timerId); // timer cancel ho gaya
```

`clearInterval(id)` — chal raha `setInterval` rokta hai.

```
let count = 1;
let intervalId = setInterval(() => {
  console.log("Count: " + count);
  count++;

  if (count > 5) {
    clearInterval(intervalId); // 5 ke baad rok do
    console.log("Interval band ho gaya");
  }
}, 1000);

// Count: 1
// Count: 2
// Count: 3
// Count: 4
// Count: 5
// Interval band ho gaya
```

💡 **Tip:** `setInterval` use kar rahe ho toh `clearInterval` lagana mat bhoolo, warna wo hamesha chalta rahega aur memory leak / performance issue ho sakta hai.

Callback patterns

Async kaam me ek common pattern hai: **"jab kaam ho jaaye, callback chala do"**. Database, file, network — har jagah.

```
function dataLao(callback) {
  console.log("Server se data laa raha hoon...");

  setTimeout(() => {
    let data = { id: 1, naam: "Aman" };
    callback(data); // kaam complete hua, ab callback bulao
  }, 2000);
}

dataLao((result) => {
  console.log("Data mil gaya:", result);
});

// Server se data laa raha hoon...
// (2 sec baad) Data mil gaya: { id: 1, naam: 'Aman' }
```

Error-first callbacks (Node.js style)

Industry me ek standard pattern hai: callback ka **pehla argument error** hota hai, aur **doosra data**.

```
function dataLao(callback) {
  setTimeout(() => {
    let error = null;
    let data = { id: 1, naam: "Aman" };

    if (error) {
      callback(error, null);
    } else {
      callback(null, data); // pehla null = koi error nahi
    }
  }, 1000);
}

dataLao((err, data) => {
```

```

if (err) {
  console.log("Error aaya:", err);
  return;
}
console.log("Success:", data);
});

```

Pattern: `callback(error, data)`. Pehle hamesha error check karo, fir data use karo.

Callback Hell (Pyramid of Doom) — THE PROBLEM 🙄

Problem tab aati hai jab ek async kaam ke baad doosra, fir teesra... aise dependent kaam karne padte hain. Har kaam ke callback ke andar agla callback ghusana padta hai, aur code **daaye taraf jhukta hua pyramid** ban jaata hai.

Real example: order do → payment karo → order confirm karo → email bhejo. Har step pichle pe depend karta hai.

```

getUser(1, (user) => {
  getOrders(user.id, (orders) => {
    getOrderDetails(orders[0], (details) => {
      processPayment(details, (payment) => {
        sendConfirmation(payment, (result) => {
          console.log("Sab ho gaya:", result);
          // 🤪 Aur andar... aur andar... aur andar...
        });
      });
    });
  });
});

```

Dekho ye triangle/pyramid shape — isi ko **"Pyramid of Doom"** ya **"Callback Hell"** kehte hain.

Iske problems:

1. **Padhna mushkil** — code daaye bhaagta jaata hai.
2. **Error handling nightmare** — har level pe alag error handle karna padta hai.

3. **Maintain karna mushkil** — beech me kuch add/change karna ho toh dimaag ghoom jaata hai.
4. **Debug karna painful.**

Isi problem ko solve karne ke liye **Promises** aaye. Chalo dekhte hain. 

3. Promises

Promise kya hai?

Promise ek object hai jo ek async operation ke **future result (ya error)** ko represent karta hai.

Real life analogy: Aapne online order kiya. Aapko turant ek **"order confirmed"** ka **promise** mil jaata hai. Ye promise abhi:

- **pending** hai (order on the way),
- baad me **deliver** ho sakta hai (success), ya
- **cancel** ho sakta hai (failure).

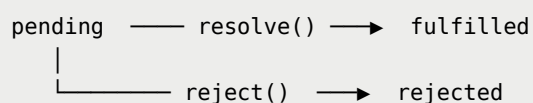
Promise ek **"main future me result dunga (ya error dunga)"** ka vaada hai.

Promise ki 3 States

Ek promise hamesha in teen states me se ek me hota hai:

State	Matlab
pending	Abhi kaam chal raha hai, na success na fail. (Initial state)
fulfilled	Kaam successfully complete (resolve ho gaya).
rejected	Kaam fail ho gaya (error aa gaya).

Important: Ek baar promise **fulfilled** ya **rejected** ho gaya, toh wo phir kabhi change nahi hota. Isko **"settled"** kehte hain. Promise sirf ek baar settle hota hai.



new Promise se promise banana

```
const myPromise = new Promise((resolve, reject) => {
  // ye "executor function" hai – turant chalta hai
  let kaamSafal = true;

  if (kaamSafal) {
    resolve("Kaam ho gaya! 🎉"); // fulfilled
  } else {
    reject("Kuch gadbad hui 😞"); // rejected
  }
});
```

- `new Promise()` ek function leta hai jise **executor** kehte hain.
- Executor ko 2 functions milte hain: `resolve` (success ke liye) aur `reject` (failure ke liye).
- `resolve(value)` call karo toh promise `fulfilled` ho jaata hai uss value ke saath.
- `reject(error)` call karo toh promise `rejected` ho jaata hai.

Async example:

```
function dataLao() {
  return new Promise((resolve, reject) => {
    console.log("Data laa raha hoon...");

    setTimeout(() => {
      let success = true;

      if (success) {
        resolve({ id: 1, naam: "Aman" });
      } else {
        reject("Server down hai!");
      }
    }, 2000);
  });
}
```

```
});  
}
```

Promise consume karna — `.then`, `.catch`, `.finally`

Promise ka result lene ke liye hum `.then()`, `.catch()` aur `.finally()` lagaate hain.

- `.then(callback)` — promise `fulfilled` hone par chalta hai, result deta hai.
- `.catch(callback)` — promise `rejected` hone par chalta hai, error deta hai.
- `.finally(callback)` — promise settle hone par hamesha chalta hai (chahe success ho ya fail). Cleanup ke liye useful (jaise loading spinner band karna).

```
dataLao()  
  .then((data) => {  
    console.log("Success:", data);  
  })  
  .catch((error) => {  
    console.log("Error:", error);  
  })  
  .finally(() => {  
    console.log("Kaam khatam (chahe pass ho ya fail)");  
  });  
  
// Data laa raha hoon...  
// (2 sec baad)  
// Success: { id: 1, naam: 'Aman' }  
// Kaam khatam (chahe pass ho ya fail)
```

Ab callback hell se compare karo — kitna saaf-suthra hai! 🥰

Promise Chaining

Yahan asli magic hai. `.then()` **ek naya promise return karta hai**, isliye aap `.then()` ke baad `.then()` laga sakte ho — ek chain ki tarah. Jo bhi `.then` se `return` karoge, wo agle `.then` me as input aata hai.

```

function add(num) {
  return new Promise((resolve) => {
    setTimeout(() => resolve(num + 10), 1000);
  });
}

add(0)
  .then((result) => {
    console.log(result); // 10
    return add(result); // naya promise return kiya
  })
  .then((result) => {
    console.log(result); // 20
    return add(result);
  })
  .then((result) => {
    console.log(result); // 30
  });

// (1s) 10
// (2s) 20
// (3s) 30

```

Ab callback hell wala example Promise chaining se — flat aur readable:

```

getUser(1)
  .then((user) => getOrders(user.id))
  .then((orders) => getOrderDetails(orders[0]))
  .then((details) => processPayment(details))
  .then((payment) => sendConfirmation(payment))
  .then((result) => console.log("Sab ho gaya:", result))
  .catch((error) => console.log("Kahin bhi error aaya toh yah an:", error));

```

Pyramid khatam! Code seedha neeche jaata hai, daaye nahi. ✅

Error propagation in chains

Promise chain ki sabse achhi baat: **ek hi `.catch()` poori chain ke saare errors pakad leta hai**. Chain me kahin bhi error aaya (ya koi promise reject hua), toh wo seedha neeche `.catch()` tak "fall through" kar jaata hai, beech ke `.then` skip karke.

```
add(0)
  .then((result) => {
    console.log(result); // 10
    throw new Error("Beech me error! 💣");
  })
  .then((result) => {
    // YE SKIP HO JAYEGA, kyunki upar error aa gaya
    console.log("Ye nahi chalega:", result);
  })
  .then((result) => {
    console.log("Ye bhi nahi chalega");
  })
  .catch((error) => {
    console.log("Error pakda:", error.message); // Beech me e
    rror! 💣
  });

// 10
// Error pakda: Beech me error! 💣
```

Yaad rakho: Ek single `.catch()` end me lagao — wo poori chain ka error handler ban jaata hai. Har `.then` ke saath alag error handling ki zaroorat nahi.

Promise Combinators — multiple promises ek saath

Bahut baar humein ek se zyada promises ek saath handle karne hote hain. JS ne 4 helpful methods diye hain:

Promise.all — sab successful hone chahiye

Multiple promises ka array leta hai. **Jab SAARE fulfill ho jaate hain**, tab ek array of results deta hai. **Agar koi ek bhi reject ho gaya**, toh poora `Promise.all` turant reject

ho jaata hai.

```
const p1 = Promise.resolve("Chai");
const p2 = Promise.resolve("Samosa");
const p3 = Promise.resolve("Biscuit");

Promise.all([p1, p2, p3])
  .then((results) => {
    console.log(results); // ['Chai', 'Samosa', 'Biscuit']
  })
  .catch((err) => {
    console.log("Koi ek fail hua:", err);
  });
```

Use case: 3 alag API calls ek saath, sabka data chahiye tabhi aage badhna hai. "All or nothing".

```
// Agar ek bhi fail ho:
const p1 = Promise.resolve("Chai");
const p2 = Promise.reject("Samosa khatam!");
const p3 = Promise.resolve("Biscuit");

Promise.all([p1, p2, p3])
  .then((results) => console.log(results))
  .catch((err) => console.log("Fail:", err)); // Fail: Samosa khatam!
```

Promise.race — jo pehle settle ho

Jo bhi promise **sabse pehle settle (fulfill YA reject)** hota hai, uska result/error de deta hai. Race (daud) jeetne wala. 🏁

```
const fast = new Promise((res) => setTimeout(() => res("Fast 🏁"), 1000));
const slow = new Promise((res) => setTimeout(() => res("Slow 🐢"), 3000));
```

```
Promise.race([fast, slow])
  .then((winner) => console.log(winner)); // Fast 🐇 (1 sec b
aad)
```

Use case: Timeout banana — "agar 5 sec me API na aaye toh fail kar do".

Promise.allSettled — sabka result chahiye, fail ho ya pass

Saare promises settle hone ka wait karta hai, aur **sabka status batata hai** — chahe wo fulfilled ho ya rejected. Ye **kabhi reject nahi hota**.

```
const p1 = Promise.resolve("Pass 1");
const p2 = Promise.reject("Fail 2");
const p3 = Promise.resolve("Pass 3");

Promise.allSettled([p1, p2, p3])
  .then((results) => console.log(results));

/*
[
  { status: 'fulfilled', value: 'Pass 1' },
  { status: 'rejected', reason: 'Fail 2' },
  { status: 'fulfilled', value: 'Pass 3' }
]
*/
```

Use case: Jab aapko har kaam ka result chahiye, chahe kuch fail ho jaayein. (e.g., 10 users ki profile load karo, jo load ho jaaye dikha do, jo fail ho usko skip karo.)

Promise.any — pehla SUCCESS chahiye

Jo bhi promise **sabse pehle FULFILL** ho jaaye, uska result deta hai. Rejections ko ignore karta hai. Agar **saare reject ho jaayein**, tab `AggregateError` deta hai.

```
const p1 = Promise.reject("Fail 1");
const p2 = new Promise((res) => setTimeout(() => res("Success
```

```
2!"), 1000));
const p3 = Promise.reject("Fail 3");

Promise.any([p1, p2, p3])
  .then((result) => console.log(result)); // Success 2! (rejections ignore)
```

`race` vs `any` ka difference:

- `race` = jo pehle **settle** ho (chahe fail ho ya pass).
- `any` = jo pehle **fulfill (success)** ho, fails ko ignore karta hai.

Quick comparison table

Method	Kab resolve hota hai	Kab reject hota hai
<code>Promise.all</code>	Jab sab fulfill	Jab koi ek reject
<code>Promise.race</code>	Jo pehle settle ho (success)	Jo pehle settle ho (fail)
<code>Promise.allSettled</code>	Hamesha (sabka status)	Kabhi reject nahi
<code>Promise.any</code>	Jo pehle fulfill ho	Jab saare reject hon

4. async / await ✨

`async` / `await` Promises ke upar ek **syntactic sugar** hai. Ye async code ko bilkul **synchronous jaisa dikhata** hai — padhne aur likhne me sabse aasaan. Andar Promises hi chal rahe hote hain.

Syntax aur Rules

Rule 1: `await` ka use karne ke liye function ko `async` banana zaroori hai.

Rule 2: `async` function **hamesha ek Promise return karta hai**.

Rule 3: `await` Promise ke settle hone ka wait karta hai aur uski **resolved value** nikaal ke deta hai.

```
function dataLao() {
  return new Promise((resolve) => {
    setTimeout(() => resolve("Data aa gaya 📦"), 2000);
  });
}
```

```

    });
  }

  // async function
  async function main() {
    console.log("Start");
    const data = await dataLao(); // yahan ruk ke wait karega
    console.log(data);             // Data aa gaya 📦
    console.log("End");
  }

  main();

  // Start
  // (2 sec baad)
  // Data aa gaya 📦
  // End

```

`.then()` wala version vs `async/await` — dono same kaam karte hain:

```

// .then() version
dataLao().then((data) => console.log(data));

// async/await version (zyada readable)
async function show() {
  const data = await dataLao();
  console.log(data);
}

```

💡 `await` sirf `async` function ke andar kaam karta hai. (Modern JS me top-level `await` modules me allowed hai, par function ke andar wala rule yaad rakho.)

Error handling — `try` / `catch`

`.then().catch()` ki jagah, `async/await` me hum normal `try` / `catch` use karte hain — bilkul synchronous code jaisa.

```

function dataLao() {
  return new Promise((resolve, reject) => {
    setTimeout(() => reject("Server down! 🤖"), 1000);
  });
}

async function main() {
  try {
    const data = await dataLao();
    console.log("Success:", data);
  } catch (error) {
    console.log("Error pakda:", error); // Error pakda: Server down! 🤖
  } finally {
    console.log("Cleanup ho gaya");
  }
}

main();

// (1 sec baad)
// Error pakda: Server down! 🤖
// Cleanup ho gaya

```

Dekho — `try/catch/finally` exactly `.then/.catch/.finally` jaisa kaam karta hai, par padhne me natural lagta hai.

Sequential vs Parallel execution ⚠️ (Bahut Important!)

Ye concept interview me aur real code dono me bahut important hai.

Sequential (ek ke baad ek) — SLOW agar kaam independent hon

```

function kaam(naam, time) {
  return new Promise((resolve) => {
    setTimeout(() => resolve(`${naam} done`), time);
  });
}

```

```

    });
  }

  async function sequential() {
    console.time("sequential");

    const a = await kaam("A", 2000); // 2 sec wait
    const b = await kaam("B", 2000); // fir 2 sec aur wait
    const c = await kaam("C", 2000); // fir 2 sec aur wait

    console.log(a, b, c);
    console.timeEnd("sequential"); // ~6000 ms 😱
  }

  sequential();

```

Yahan har `await` apne se pehle wale ka wait karta hai. Total **6 second** — bekaar, kyunki teeno kaam ek doosre pe depend nahi karte!

Parallel (sab ek saath) — FAST

Agar kaam independent hain, toh unhe ek saath start karo aur `Promise.all` se wait karo.

```

  async function parallel() {
    console.time("parallel");

    // Teeno turant start kar diye (await nahi laga abhi)
    const pa = kaam("A", 2000);
    const pb = kaam("B", 2000);
    const pc = kaam("C", 2000);

    // Ab sabka result ek saath wait karo
    const [a, b, c] = await Promise.all([pa, pb, pc]);

    console.log(a, b, c);
  }

```

```

    console.timeEnd("parallel"); // ~2000 ms 🚀
  }

  parallel();

```

Total sirf **2 second**! Kyunki teeno kaam saath-saath chale.

🎯 **Golden Rule:** Agar kaam ek doosre pe **depend** karte hain → sequential (`await` ek ke baad ek). Agar **independent** hain → parallel (`Promise.all`). Galat choice se app slow ho jaata hai.

Common Mistakes 🚨

Mistake 1: `await` lagana bhool jaana

```

async function galat() {
  const data = dataLao(); // ❌ await missing!
  console.log(data); // Promise { <pending> } – data nahi, Promise object mila!
}

async function sahi() {
  const data = await dataLao(); // ✅ await lagaya
  console.log(data); // asli data
}

```

`await` bina aapko actual value nahi, **pending Promise object** milta hai. Bahut common bug.

Mistake 2: `try/catch` lagana bhool jaana

```

// ❌ Galat – agar reject hua toh "unhandled promise rejection" crash
async function galat() {
  const data = await dataLao(); // agar fail hua toh app crash
}

```

```

    console.log(data);
  }

  // ✅ Sahi – error handle kiya
  async function sahi() {
    try {
      const data = await dataLao();
      console.log(data);
    } catch (e) {
      console.log("Handle kiya:", e);
    }
  }
}

```

Mistake 3: Loop me galat tarike se `await` (forEach trap)

`forEach` ke andar `await` kaam nahi karta jaise aap sochte ho — wo wait nahi karta!

```

// ❌ Galat – forEach await ka wait nahi karta
async function galat(items) {
  items.forEach(async (item) => {
    await process(item); // ye sab ek saath fire ho jaayenge,
    // order nahi rahega
  });
  console.log("Done"); // ye sabse pehle print ho jaayega! galat
}

// ✅ Sahi – for...of use karo agar sequence chahiye
async function sahi(items) {
  for (const item of items) {
    await process(item); // ye sahi se ek-ek karke wait karega
  }
  console.log("Done"); // sahi me last me
}

```

Mistake 4: Independent kaam ko sequentially await karna (slow)

Ye upar wala "sequential vs parallel" wala point hai — independent kaam ko

`Promise.all` se parallel karo, ek-ek `await` se nahi.

5. The Event Loop 🎉 (The Big "WOW" Moment)

Ye section JS ka sabse important concept hai. Isko samajh gaye toh aap JS me kaafi aage nikal gaye. Dhyaan se padhna.

Sawaal: JS single-threaded hai (ek hi kaam ek time pe), toh fir wo async kaam (timers, network) ek saath kaise handle karta hai bina block kiye? 😞

Jawaab: Event Loop ki wajah se. Chalo har piece samajhte hain.

Components — pehle ye 4 cheezein samajho

1. Call Stack (revisited)

JS jahan code actually run karta hai. Ye ek **stack** (LIFO — Last In First Out) hai. Jab function call hota hai, wo stack pe **push** hota hai. Jab return karta hai, **pop** ho jaata hai.

```
function teen() { console.log("teen"); }
function do_() { teen(); }
function ek() { do_(); }
ek();

// Call stack ka flow:
// ek() push
//   do_() push
//     teen() push → "teen" print → teen() pop
//   do_() pop
// ek() pop
```

Important: Call stack pe ek time pe ek hi kaam chalta hai. Agar yahan koi lamba (blocking) kaam aa gaya toh sab ruk jaata hai.

2. Web APIs / Browser APIs

`setTimeout`, `fetch`, DOM events — ye sab **JS ka part nahi hain!** Ye **browser** provide karta hai (Node.js me C++ APIs). Jab aap `setTimeout` call karte ho, JS usko **browser ko de deta hai** aur aage badh jaata hai. Browser us timer ko side me handle karta hai. Isi wajah se JS block nahi hota.

3. Callback Queue (Macrotask Queue)

Jab Web API ka kaam complete ho jaata hai (jaise timer khatam, ya event fire hua), toh uska callback yahan **Callback Queue** (ya Macrotask Queue / Task Queue) me aake line me lag jaata hai. Ye ek **queue** (FIFO — First In First Out) hai.

Macrotasks me aate hain: `setTimeout`, `setInterval`, `setImmediate`, I/O, UI events.

4. Microtask Queue

Ek **alag aur zyada priority** wali queue. Isme aate hain:

- **Promise** callbacks (`.then`, `.catch`, `.finally`)
- `await` ke baad wala code
- `queueMicrotask()`

Sabse important rule: Microtask queue ki priority macrotask (callback) queue se **HAMESHA** zyada hai. ★

Event Loop ka kaam kya hai?

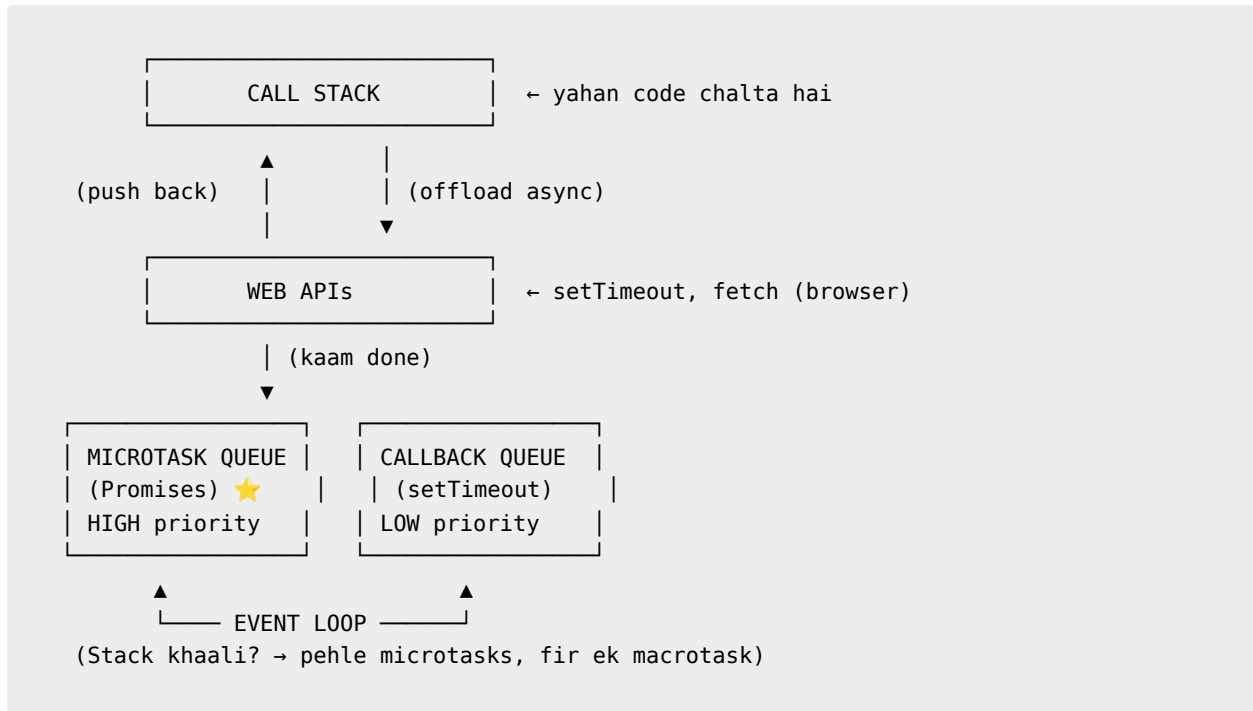
Event Loop ek watchman ki tarah continuously check karta rehta hai:

"Kya Call Stack khaali hai? Agar haan, toh pehle saara **Microtask Queue** khaali karo (poora), fir **Callback (Macrotask) Queue** se ek kaam uthao aur stack pe daalo."

Steps:

1. Synchronous code pehle poora chalao (Call Stack pe).
2. Stack khaali hone par, **saare microtasks** chalao (poori microtask queue khatam karo).
3. Fir **ek macrotask** uthao.
4. Phir wapas step 2 (microtasks check karo), fir agla macrotask...

5. Ye loop chalta rehta hai. Isiliye naam — **Event Loop**.



Step-by-step example

```
console.log("1");

setTimeout(() => console.log("2"), 0);

Promise.resolve().then(() => console.log("3"));

console.log("4");
```

Output: 1, 4, 3, 2

Chalo trace karte hain:

1. `console.log("1")` → sync → turant **1** print.
2. `setTimeout(...)` → browser ko diya. Timer (0ms) ke baad callback **Macrotask queue** me jaayega.
3. `Promise.resolve().then(...)` → callback **Microtask queue** me jaata hai.

4. `console.log("4")` → sync → turant **4** print.
5. Sync code khatam. Call stack khaali.
6. Event loop: **pehle microtasks** → `.then` callback chala → **3** print.
7. Ab macrotask → `setTimeout` callback → **2** print.

Microtask (3) ne macrotask (2) ko beat kiya — chahe setTimeout 0 ms hi kyun na ho! Yahi event loop ka magic hai. ✨

Tricky Output Prediction Problems 🧩

Interview me ye type ke questions pakka aate hain. Practice karo!

Problem 1

```
console.log("A");

setTimeout(() => console.log("B"), 0);

Promise.resolve()
  .then(() => console.log("C"))
  .then(() => console.log("D"));

console.log("E");
```

<details>

<summary>Answer (try karo pehle, fir kholo)</summary>

Output: `A, E, C, D, B`

- Sync: `A, E`.
- Microtasks: `C` chala. `.then` ke baad ka `.then` ek naya microtask banata hai → `D`. (Microtask queue ek baar me poori khatam hoti hai, naye microtasks bhi include.)
- Macrotask: `B`.

</details>

Problem 2

```
console.log("Start");

setTimeout(() => console.log("Timeout 1"), 0);

Promise.resolve().then(() => {
  console.log("Promise 1");
  setTimeout(() => console.log("Timeout 2"), 0);
});

setTimeout(() => console.log("Timeout 3"), 0);

console.log("End");
```

<details>

<summary>Answer</summary>

Output: Start, End, Promise 1, Timeout 1, Timeout 3, Timeout 2

- Sync: Start, End.
- Microtask: Promise 1 (aur iske andar ek naya setTimeout schedule hua jo Timeout 2 ke liye microtask banata hai — par queue me sabse last me).
- Macrotasks order: Timeout 1, Timeout 3 (ye pehle se queue me the), fir Timeout 2 (baad me add hua).

</details>

Problem 3 (async/await wala — thoda advanced)

```
async function async1() {
  console.log("async1 start");
  await async2();
  console.log("async1 end"); // ye await ke baad — microtask
  ban jaata hai
}
```

```

async function async2() {
  console.log("async2");
}

console.log("script start");

async1();

console.log("script end");

```

<details>

<summary>Answer</summary>

Output:

```

script start
async1 start
async2
script end
async1 end

```

- `console.log("script start")` → sync.
- `async1()` call → "async1 start" print → `await async2()` → "async2" print. `await` ke baad ka code (`async1 end`) microtask ban ke ruk jaata hai.
- `console.log("script end")` → sync.
- Sync khatam → microtask chala → `async1 end`.

Key insight: `await` ke baad ka pura code ek microtask ki tarah behave karta hai — wo current sync code ke baad chalta hai.

</details>

 **Rule yaad rakho:** Sync code → fir saare Microtasks (Promises, await ke baad) → fir Macrotasks (setTimeout). Hamesha isi order me.

6. Fetch API aur HTTP Requests

Ab tak humne fake/timer wale async kaam dekhe. Ab **real** network calls — server se actual data laana.

HTTP requests ka basic idea

Jab aap kisi website pe jaate ho, aapka browser server ko ek **HTTP request** bhejta hai, aur server **response** deta hai. Common request types (methods):

- **GET** — data laana (read)
- **POST** — naya data bhejna (create)
- **PUT/PATCH** — data update karna
- **DELETE** — data delete karna

`fetch()` — built-in function

`fetch(url)` ek **Promise** return karta hai. Isliye iske saath hum `.then` ya `async/await` use karte hain.

`.then` version

```
fetch("<https://jsonplaceholder.typicode.com/users/1>")
  .then((response) => response.json()) // response ko JSON me
  convert karo (ye bhi promise hai)
  .then((data) => console.log(data))
  .catch((error) => console.log("Error:", error));
```

Important do-step process:

1. `fetch()` → `response` object deta hai (abhi raw data nahi).
2. `response.json()` → response body ko JS object me parse karta hai (ye **bhi** ek promise return karta hai, isliye iska bhi `await` / `.then` chahiye).

`async/await` version (recommended — clean)

```
async function userLao() {
  try {
    const response = await fetch("<https://jsonplaceholder.ty
```

```

picode.com/users/1>");

    // Check karo request successful thi ya nahi
    if (!response.ok) {
        throw new Error(`HTTP error! Status: ${response.status}`);
    }

    const data = await response.json();
    console.log("User mil gaya:", data);
} catch (error) {
    console.log("Kuch galat hua:", error.message);
}
}

userLao();

```

⚠ **Bahut Important gotcha:** `fetch` sirf tab reject hota hai jab **network fail** ho (internet nahi hai). Agar server **404 ya 500** error deta hai, toh bhi `fetch` ka promise **resolve hota hai** (reject nahi)! Isliye `response.ok` ya `response.status` khud check karna zaroori hai. Ye beginners bahut miss karte hain.

POST request — data bhejna

```

async function userBanao() {
    try {
        const response = await fetch("<https://jsonplaceholder.ty
picode.com/users>", {
            method: "POST", // request type
            headers: {
                "Content-Type": "application/json", // bata rahe hain
                ki JSON bhej rahe hain
            },
            body: JSON.stringify({ // JS object ko JSON string
                me convert
            }

```

```

        name: "Rahul",
        email: "rahul@example.com",
    })),
});

const data = await response.json();
console.log("Ban gaya:", data);
} catch (error) {
    console.log("Error:", error.message);
}
}

userBanao();

```

Dhyaan do:

- `method: "POST"` — kis tarah ki request.
- `headers` — server ko batate hain ki kis format me data bhej rahe.
- `body` — actual data, jise `JSON.stringify()` se string banana padta hai (network par data string me jaata hai).

Isi JSON ke baare me ab detail me baat karte hain. [↓](#)

7. JSON, `JSON.parse`, `JSON.stringify`

JSON kya hai?

JSON = JavaScript Object Notation. Ye data ko **text (string)** format me likhne ka ek standard tareeka hai. Servers aur clients ke beech data is format me hi travel karta hai, kyunki network par sirf text hi bhej sakte hain (objects directly nahi).

JSON dikhne me JS object jaisa hai, par kuch rules hain:

- **Keys hamesha double quotes me** honi chahiye: `"name"` (single quote ya bina quote nahi chalega).
- **Strings hamesha double quotes me.**
- Values ho sakti hain: string, number, boolean, null, array, ya doosra object.

- **Functions**, `undefined` **allowed nahi** hote JSON me.

```
{
  "name": "Rahul",
  "age": 25,
  "isStudent": true,
  "skills": ["JS", "React"],
  "address": {
    "city": "Indore",
    "pin": 452001
  }
}
```

JS object vs JSON: JS object me keys bina quote ke aur single quote me chalti hain. JSON me **strictly double quotes** chahiye. JSON ek **string** hai, JS object ek actual object hai.

JSON.stringify() — Object → String

JS object/array ko **JSON string** me convert karta hai. (Server ko data bhejte waqt zaroori — jaise upar POST request me dekha.)

```
const user = {
  name: "Rahul",
  age: 25,
  skills: ["JS", "React"],
};

const jsonString = JSON.stringify(user);

console.log(jsonString);
// {"name":"Rahul","age":25,"skills":["JS","React"]}

console.log(typeof jsonString); // string ✓
```

Pretty printing (readable format) ke liye 3rd argument me indentation:

```

console.log(JSON.stringify(user, null, 2));
/*
{
  "name": "Rahul",
  "age": 25,
  "skills": [
    "JS",
    "React"
  ]
}
*/

```

Note: `JSON.stringify` functions aur `undefined` ko skip kar deta hai:

```

const obj = {
  naam: "Aman",
  greet: function () {}, // ye skip ho jaayega
  age: undefined,         // ye bhi skip
};
console.log(JSON.stringify(obj)); // {"naam":"Aman"}

```

`JSON.parse()` — String → Object

JSON string ko wapas usable JS object me convert karta hai. (Server se data aane par zaroori — jaise `response.json()` andar yahi karta hai.)

```

const jsonString = '{"name":"Rahul","age":25,"skills":["J
S","React"]}';

const user = JSON.parse(jsonString);

console.log(user);           // { name: 'Rahul', age: 25, skill
s: [ 'JS', 'React' ] }
console.log(user.name);      // Rahul

```

```
console.log(user.skills[0]); // JS
console.log(typeof user); // object ✓
```

Galat JSON parse karne par error:

```
try {
  JSON.parse("{name: 'Rahul'}"); // ✗ keys/strings double-quotes me nahi
} catch (e) {
  console.log("Invalid JSON:", e.message);
}
// Hamesha parse ko try/catch me rakho agar source pe bharosa nahi
```

stringify ⇔ **parse** ek doosre ke ulte hain

```
const original = { a: 1, b: [2, 3] };

const str = JSON.stringify(original); // object → string
const back = JSON.parse(str);         // string → object

console.log(back); // { a: 1, b: [ 2, 3 ] } – wapas wahi
```

Bonus: Deep copy ka jugaad

`JSON.parse(JSON.stringify(obj))` ek purana trick hai object ki **deep copy** banane ka (nested objects bhi copy ho jaate hain). Limitation: functions, `undefined`, dates theek se copy nahi hote. Simple data ke liye chalta hai.

```
const original = { naam: "Aman", address: { city: "Indore" } };
const copy = JSON.parse(JSON.stringify(original));

copy.address.city = "Bhopal";
```

```
console.log(original.address.city); // Indore (original safe! 🎉)
console.log(copy.address.city);      // Bhopal
```

Quick Revision / Cheat Sheet

Concept	Ek line summary
Synchronous	Line by line, ek kaam khatam tabhi agla. Blocking.
Asynchronous	Lamba kaam side me, code rukta nahi. Non-blocking.
Callback	Function jo dusre function ko pass hota hai, baad me call hone ke liye.
Callback Hell	Nested callbacks ka pyramid — unreadable mess.
Promise	Future result/error ka object. States: pending → fulfilled/rejected.
.then/.catch/.finally	Success / error / always — promise consume karne ke liye.
Promise.all	Sab pass tabhi success; ek fail toh poora fail.
Promise.race	Jo pehle settle ho (pass ya fail).
Promise.allSettled	Sabka result, fail ho ya pass. Kabhi reject nahi.
Promise.any	Jo pehle pass ho; saare fail tabhi error.
async/await	Promises ka clean syntax. await = result ka wait.
try/catch	async/await me error handling.
Event Loop	Sync → saare Microtasks → ek Macrotask → repeat.
Microtask	Promise/.then/await — HIGH priority.
Macrotask	setTimeout/setInterval — LOW priority.
fetch	Network call, promise return karta hai. <code>response.ok</code> check karo!
JSON.stringify	Object → JSON string (bhejne ke liye).
JSON.parse	JSON string → Object (lene ke baad).

Practice Questions (homework)

1. `setTimeout` aur `setInterval` ka difference apne words me likho, aur ek "countdown timer" (5 se 0 tak) banao jo end pe khud band ho jaaye

(`clearInterval`).

2. Ek callback hell example likho (kam se kam 3 levels), fir usko Promise chain me convert karo, fir `async/await` me.
3. Ye output predict karo (try/catch nahi, sirf dimaag se):

```
console.log(1);setTimeout(() => console.log(2), 0);Promise.resolve().then(() => console.log(3));Promise.resolve().then(() => { console.log(4); setTimeout(() => console.log(5), 0);});console.log(6);
```

4. `fetch` se `https://jsonplaceholder.typicode.com/posts/1` ka data lao aur title console pe print karo — `async/await` + `try/catch` ke saath.
5. Ek object banao, usko `JSON.stringify` karke string banao, fir `JSON.parse` karke wapas object banao. `typeof` se dono check karo.
6. `Promise.all` vs `Promise.allSettled` ka practical difference dikhane ke liye ek example banao jisme ek promise fail ho.

Happy coding! Doubt ho toh batch me poochna. Async JS practice se hi clear hota hai — code likho, output predict karo, fir run karke check karo. 🚀